—: Stack :—

Stack is linear type of data structure it is work on (LIFO) principal. there are two basic operation performed.

① Push ② Pop

* Push means insterting element into stack.
* pop means deleting element from stack.

Data may be push or pop at only one End of stack that end is called Top.

Conditions of stack:—

(I) If Push data into stack then

Top = Top + 1

(II) If pop the element from stack.

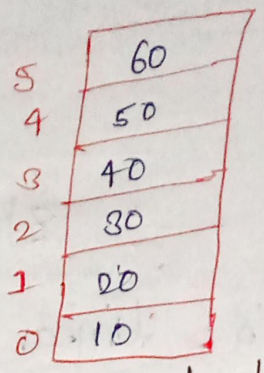
Top = Top - 1

(III) overflow Condition

Top = MAX - 1

(iv) Under flow Condition

Top = -1



① Algorithm for push element into stack:-

(i) Initialize stack 'S'
(ii) check overflow Condition. if Top = MAX-1 "print overflow Condition"
(iii) Exit
(iv) Push data element into stack
Top = Top + 1
(v) stack [Top] = element
(vi) Exit

) Algorithm for Pop element from stack:-

(i) Initialize stack 'S'
(ii) check underflow Condition if Top = -1 then print under flow Condition
(iii) Exit
v) Pop the element from stack
element

Element = Stack [Top]

(V) Top = Top - 1

(VI) exit

* Write a fenction in c to push the element.

```
Void Push( )
{   int element;
    if (Top == Max-1)
       printf("stack is overflow");
    else
    { printf ("Enter the element");
      scanf ("%d", &element);
      Top = Top+1;
      Stack [Top] = element;
    }
}
```

* Write a fenction in c to pop the element.

```
Void POP ( )
{ int element
  if (Top == -1)
     printf ("stack is underflow")
  else
    {
```

```
    element = stack[Top]
    Top = Top-1
    printf("%d has been deleted", element);
}
```

Application of stack :-

① Polish and Reverse Polish Notation :-

The method of writting operators of an expression either before their operands or after them is called the polish and Reverse polish notation.

(i) Prefix notation → +AB
(ii) Infix notation = A+B
(iii) Postfix notation. = AB+

An Arithmatic expression can represents in above three notation.

Polish notation :- In which operator symbol are placed before it's both operands.

Ex. +AB

Reverse polish notation :- In which operator symbol are placed after both operands it's known as reverse polished notation.

ex. AB+

* Priority level / Precidence level :-

( ) → highest priority

↑ or ^ → second " { ↑ - exponential, ^ - power

*, / → third "

+, - → least "

$ - ↑

Pro 1 Convert Infix form to post fix form

A - B * C

A - (B * C)

A - (BC*) { let BC* = T

A - T

AT -

ABC* - Ans

(2) Convert Infix to prefix form.

A - B * C

A - (B * C)

A - (*BC) { let *BC = T

A - T

- AT

-A*BC Ans

(3) Convert Infix into postfix form.

$(A + B) * D$

$(AB+) * D$ { let $AB+ = T$

$T * D$

$TD*$

$\boxed{AB+D*}$ Ans

Q. Convert Infix to postfix form.

$(A+B) * C/D + E \uparrow F/G$

$(AB+) * C/D + E \uparrow F/G$ { let $AB+ = P$

$P * C/D + (E \uparrow F)/G$

$P * C/D + (EF\uparrow)/G$ { let $EF\uparrow = Q$

$P * C/D + Q/G$

$(PC*)/D + Q/G$ { let $(PC*) = R$

$R/D + Q/G$

$(RD/) + Q/G$ { let $(RD/) = S$

$S + Q/G$

$S + (Q/G)$

$S + (QG/)$

~~RD/~~ $SQG/+$

$RD/QG/+$

$PC*D/EF\uparrow G/+$

$\boxed{AB+C*D/EF\uparrow G/+}$ Ans

Algorithm to Convert Infix to ~~Postfix~~ Postfix :-

Polish (Q, P)

Suppose Q is an arithmatic expression written in infix notation. This algorithm find equivalent postfix expression P.

(I) Push '(' on the stack and ')' to the end of Q.

(II) Scan Q from left to Right and Repeat steps to 6 for each elements of Q untill the stack is empty.

(III) if an operand is encountered add it to P.

(IV) if a left parenthesis is encountered push it on to stack.

(V) if an (*) is encountered then.

(a) Repeatedly pop from stack and add to P each operator (on the top of stack) which has the same precedance as or higher precedance than (*).

(b) Add (*) to stack.

(VI) if Right Parenthesis is encountered then

(a) Repeatedly pop from stack and add to P each (on the top of stack) untill left Parenthen's is encountered.

(b) Remove the left parenthesis.

(VII) Exit

(1) Convert Infix expression into postfix form.

Q = A+(B*C-(D/E↑F)*G)*H)

Q	Stack	Postfix P
A	(	A
+	(+	A
(	(+(	A
B	(+(	AB
*	(+(*	AB
C	(+(*	ABC
-	(+(-	ABC*
(	(+(-(	ABC*
D	(+(-(	ABC*D
/	(+(-(/	ABC*D
E	(+(-(/	ABC*DE
↑	(+(-(/↑	ABC*DE
F	(+(-(/↑	ABC*DEF
)	(+(-	ABC*DEF↑/
*	(+(-*	ABC*DEF↑/
G	(+(-*	ABC*DEF↑/G
)	(+	ABC*DEF↑/G*-
*	(+*	ABC*DEF↑/G*-
H	(+*	ABC*DEF↑/G*-H
)	Empty	ABC*DEF↑/G*-H*+

Ans

(2) Q = A-B/C+D*E+F

Q	Stack	Postfix P
A	(	A
-	(-	A
B	(-	AB
/	(-/	AB
C	(-/	ABC
+	(+	ABC/-
D	(+	ABC/-D
*	(+*	ABC/-D
E	(+*	ABC/-DE
+	(+	ABC/-DE*+
F	(+	ABC/-DE*+F
)	empty	ABC/-DE*+F+

Ans - ABC/-DE*+F+

Evaluation of Postfix form using stack:-

1. Add a ')' at the end of P
2. Scan P from left to Right and repeat steps 3 and 4 for each element of P untill ')' is encountered.
3. If an operand is encountered, Push it on the stack.
4. If an ⊗ is encountered then
   (a) Remove the two top element of stack where A is the top element and B is the next to top element.
   (b) Evaluate B ⊗ A
   (c) Place the Result of 4(b) back on stack.
5. Set value equal to the top element of stack.
6. Exit.

———— * ————

① Evaluation of Postfix form using stack.
Find out the value.

P = 5, 6, 2, +, *, 12, 4, /, −,)

P	stack
5	5
6	5, 6
2	5, 6, 2
+	5, 8
*	40
12	40, 12
4	40, 12, 4
/	40, 3
−	37
)	37 ~~Ans~~

A = 2, B = 6
B ⊗ A = 6 + 2 = 8

A = 8, B = 5 ⇒ 5 × 8 = 40

A = 4, B = 12 B ⊗ A = 12/4 = 3

A = 3, B = 40 B ⊗ A = 40 − 3 = 37

Applications :-

(i) stacks can be used to check paranthesis matching in an expression.

(ii) It is used in backtracking problems.

Recursion :- Function calling itself is known as recursion.

Tower of Hanoi Problem :- {Hanoi is capital Vietnam}

Rules of Tower of hanoi Problem :-

① Lets Consider 3 Pegs. A, B, C.

② Suppose Peg A is source and peg C is destination. and Peg B is via

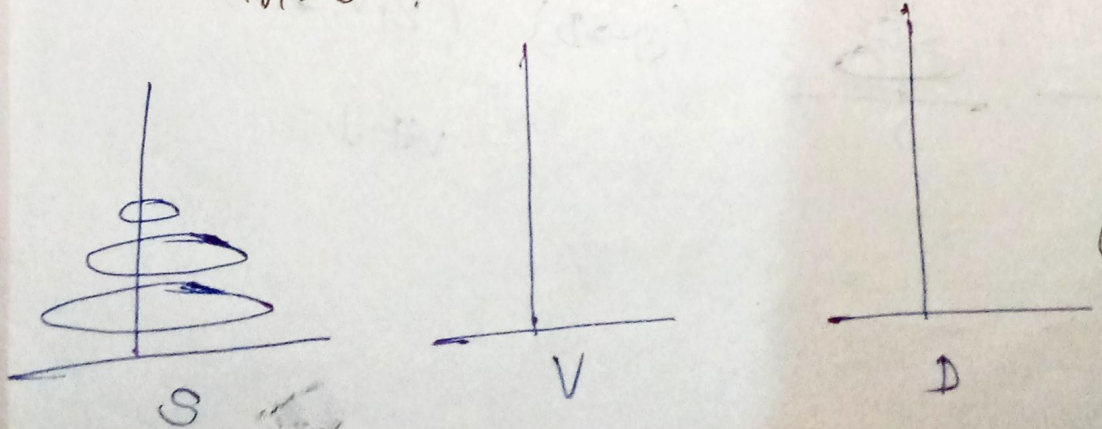
③ no. of move from source A to destination is $2^n - 1$

④ Only one disk can be moved.

⑤ Always top disk can be moved.

⑥ Always smaller disk can be take placed on larger disk.

Problem-1 solve the tower of hanoi problem for $n = 3$ disk.



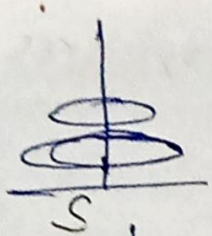
$n = 3$

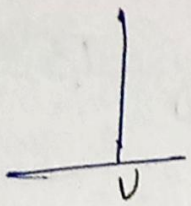
no of move

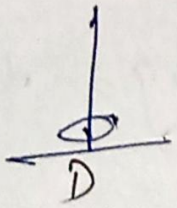
$\binom{n}{3, S, D, V}$ $2^3 - 1$

$= 7$

(I)



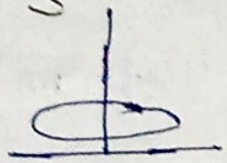


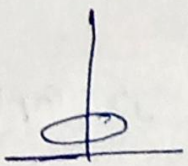


$(S \to D)$

(3.

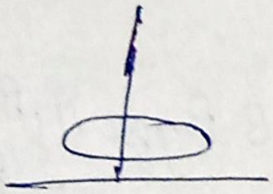
(II)

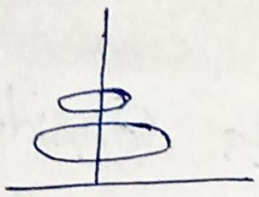




$(S \to V)$

(III)

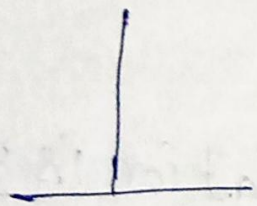


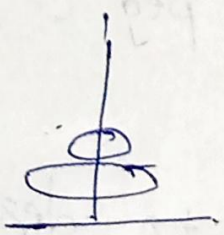


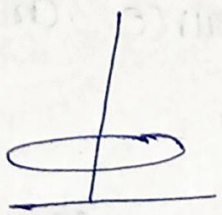
$(D \to V)$

$(n-1, S, V, D)$

(IV)





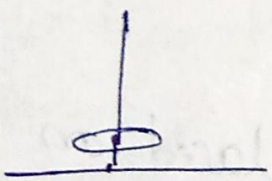


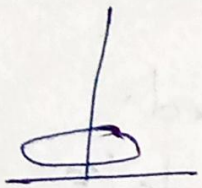
$(S \to D)$

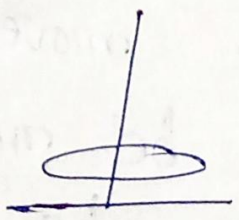
$(1, S, D, V)$

source to destination through via

(V)

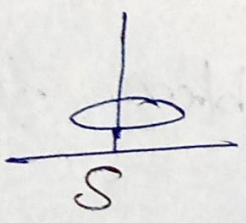


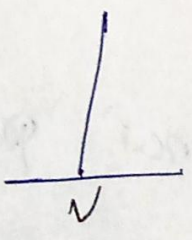


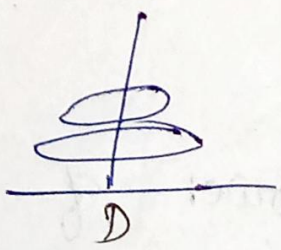


$(V \to S)$

(VI)

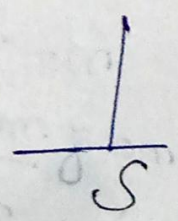


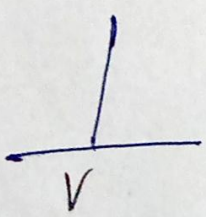


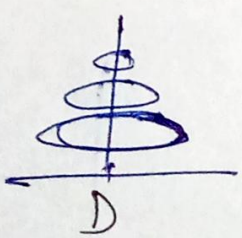


$(V \to D)$

(VII)







$(S \to D)$

$(n-1, V, D, S)$

~~1~~

(n-1, S, V, D)

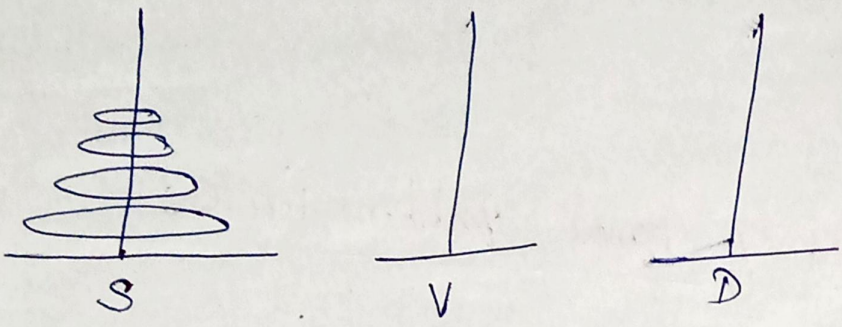
(1, S, D, V)

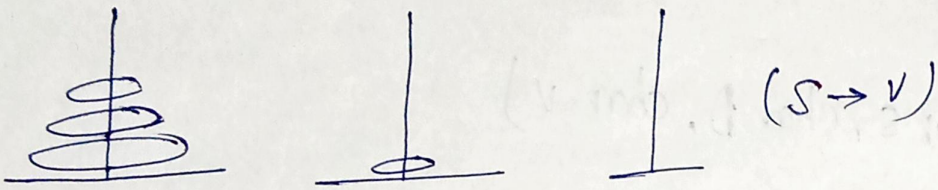
(n-1, V, D, S)

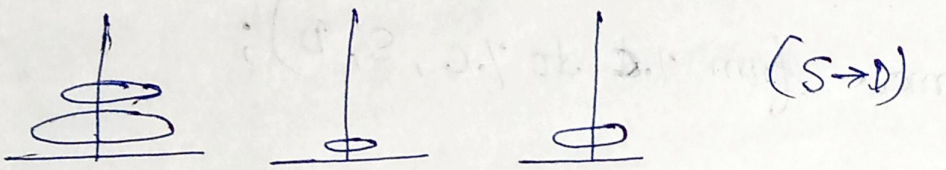
Implementation of tower of hanoi Problem in C:-

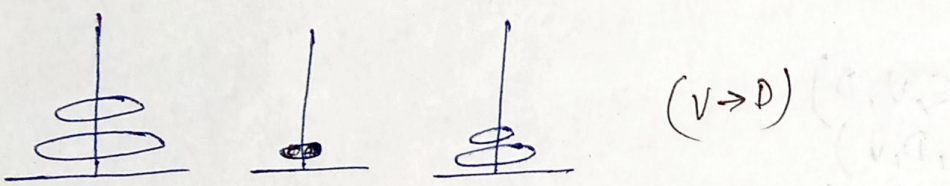
```
# include <stdio.h>
# include <conio.h>
void move (int n, char S, char D, char V)
{   if (n==1)
      printf ("\n move from %c to %c", S, D);
    else
    {
      move (n-1, S, V, D)
      move (1, S, D, V)
      move (n-1, V, D, S)
    }
}
Void main()
{   int n
    printf ("Enter no. of disk");
    Scanf ("%d", &n)
    move (n, 'A', 'C', 'B')
    getch();
}
```

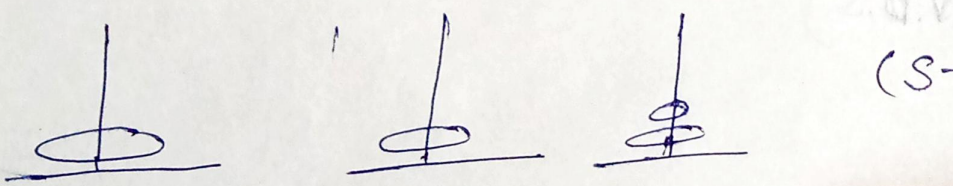
For n=4 disk:-

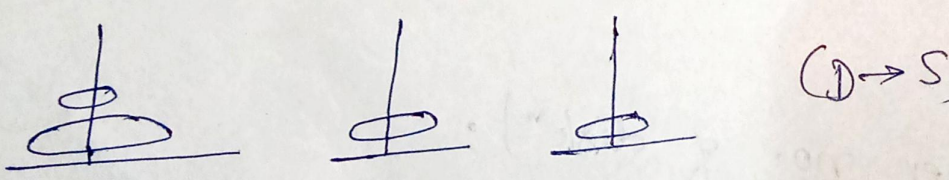


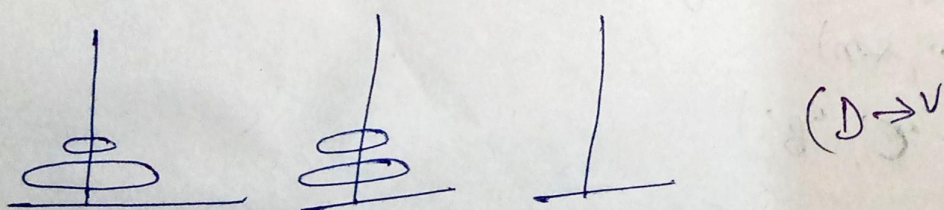


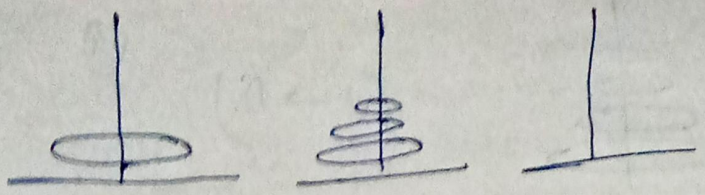




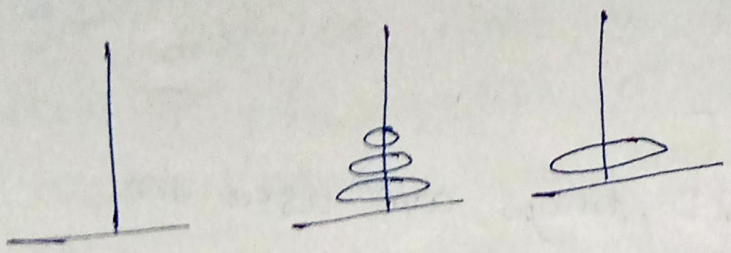




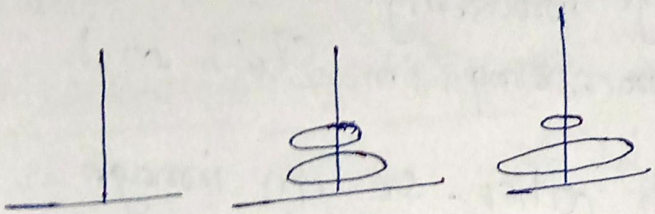




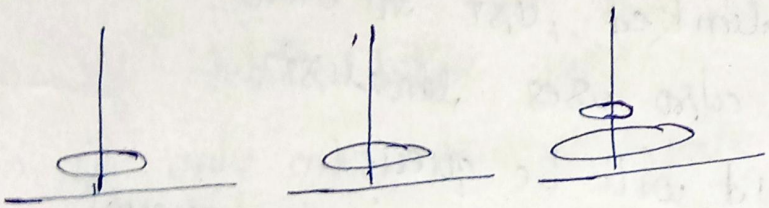
(S → V)



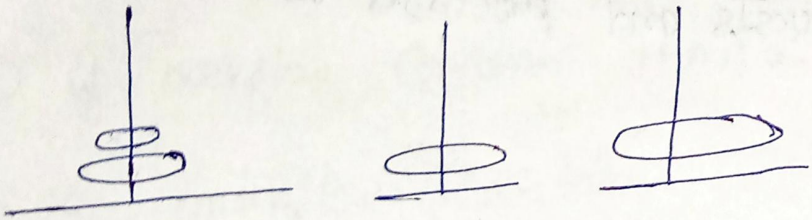
(S → D)



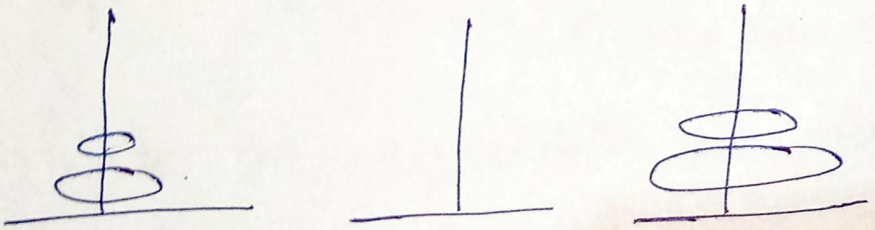
(V → D)



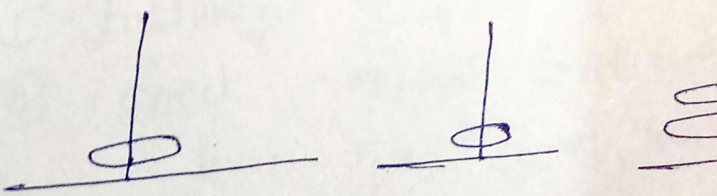
(V → S)



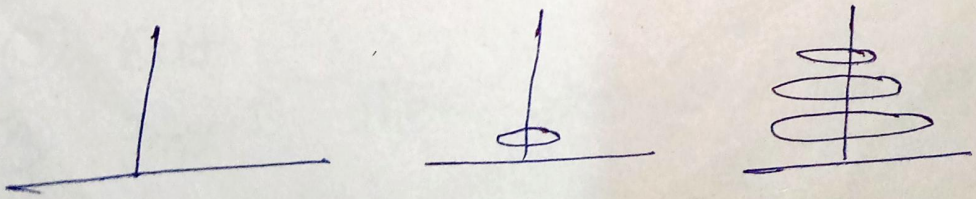
(D → S)



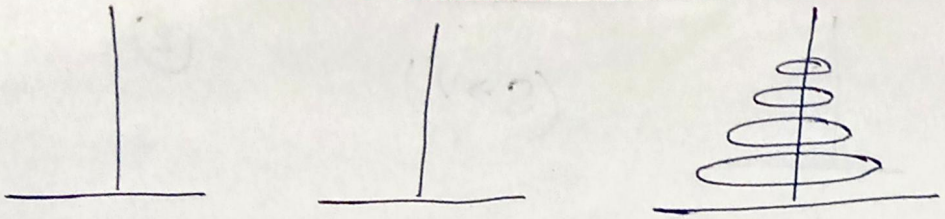
(V → D)



(S → V)



(S → D)



(N → D)

Algorithm:-

Application of data structure:-

① 2D, Arrays are used in image processing.

② It is also used in speech processing, in which each speech signal is an array.

③ image are linked with each other. so an image viewer software uses a linked list to view.

④ In the music player also uses linklist.

⑤ sending an E-mail, it will be queued.

⑥ Most of internet requests and processes uses queue.

⑦ Data organization

-: Queue :-

Queue is a linear type of data structure. It work on First in First out (FIFO) Principal. Data may be insert at one end that end is known as Rear end, and data may be delete at other end is known as Front end.

* Conditions for Queue:-

① if insert the element then
Rear = Rear + 1

② if delete the element then front = front + 1

③ if only one element into the ~~queque~~ queue.
Front = Rear

④ if overflow Condition then - Rear = MAX - 1

⑤ if underflow Condition-
Front = Rear = -1 ; ; F > R

* Write an Algorithm to insert element into queue.

① Initialize Queue 'q'
② check overflow Condition. if Rear = MAX+1
then print "overflow Condition"
③ Exit
④ Insert the element into 'queue.

if (Front == -1) then Front = 0

⑤ Rear = Rear + 1

⑥ set queue [Rear] = element

⑦ Exit.

Write a function in 'C' to insert element into queue.

```
Void Insert()
{
    int element
    if (Rear == MAX-1)
      printf("overflow");
    else
    {
      if (Front == -1)
          Front = 0
      printf("Enter a element");
      Scanf("%d", &element);
      Rear = Rear + 1;
      queue[Rear] = element;
    }
}
```

Write an Algorithm to delete element form queue.

① Initialize queue 'q'
② check underflow Condition
   if F=R=-1 || F>R then
   underflow Condition occur
③ Exit
④ delete element form queue.
   element = queue [Front]
⑤ Front = Front + 1
⑥ Exit.

Write a function in 'c' to delete element form queue

```
Void delete ()
{   int element;
    if (F=R=-1 || F>R)
     printf ("queue is underflow");
    else
    {
       element = queue [F]
       F = F+1
       printf ("%d has been deleted", element);
    }
}
```

Implementation of simple Queue using Array:-

```
#include<stdio.h>
#include<conio.h>
#define MAX 5
int queue[MAX];
int Rear=-1
int Front=-1
void Insert()
{
    int element;
    if(Rear==MAX-1)
        printf("queue is overflow");
    else
    {
        Rear=Rear+1;
        printf("Enter a Value");
        Scanf("%d",&element);
        if(Front==-1)
            Front=0;
        queue[Rear]=element;
    }
}
```

```
void delete ( )
{
    int element;
    if (Front == -1 || Front > Rear)
      printf ("queue is underflow");
    else
    {
        element = queue [Front];
        Front = Front + 1;
    }
}
void display ( )
{
    int i;
    if (Front == -1 || Front > Rear)
      printf ("No any element in queue");
    else
    {   for( i = Front ; i <= Rear ; i++ )
          printf ("%d \n", queue [i]);

    }
}
```

```
void main ( )
{
  int choice;
  do
  {
    printf ("1. Insert \n");
    printf ("2. delete \n");
    printf ("3. display \n");
    printf ("4. Exit \n");
    printf ("Enter your choice");
  while
    scanf ("%d", &choice);
    switch (choice)
    { case 1 : Insert ( );
               break;
      case 2 : delete ( );
               break;
      case 3 : display ( );
               break;
      case 4 : Exit (0);
    }
  } while (choice <= 4);
}
```

—: Circular Queue :—

Write Algorithm for Insert element in Queue.

(1) Initialize Queue Q.
(2) check ~~Under~~ overflow Condition?
    if F=(R+1)% MS
(3) Exit
(4) Insert element in to queue.
    R=(R+1) % MS
    if Front == -1 then (Front = 0)
(5) Q[R] = element
(6) Exit.

Write A function in 'C' to insert element into queue.

```
Void Insert()
{ int element;
  if (F==(R+1)%MS)
    printf("queue is overflow");
  else if(F==-1); F=0;
  { printf("Enter element");
    scanf("%d", &element);
    R= (R+1)%MS;
    queue[R] = element;
  }
}
```

Algorithm for deletion :-

(1) Initialize circular queue Q.

(2) check underflow Condition ?

if (Front == -1) then underflow Condition occur.

(3) Exit.

(4) element = Q[Front]

Front = (Front+1) % MS

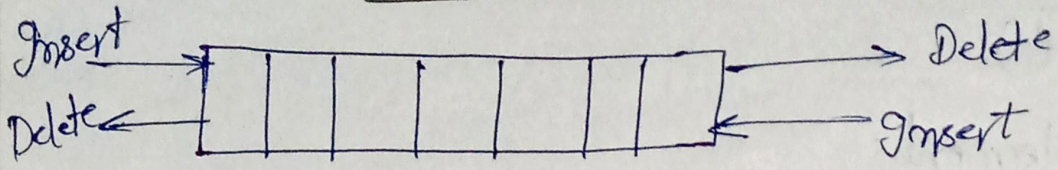
(5) Exit.

Function

Function for deletion :-

```
Void delete()
{
    int element;
    if (F==-1)
      printf("queue is underflow");
    else
    {
        element = Q[F];
        printf("%d is deleted", element);
        if (F==R)
        {
            F=-1;
            R=-1;
        }
        else
            F = (F+1) % MAX
    }
}
```

:- Double Ended Queue :-



Types of double ended queue :-

(I) I/P Restricted DE queue

(II) O/P Restricted DE queue

Types of operation performed in DE queue :-

(1) Insertion at Rear end of DE queue. - R = R+1

(2) Deletion of Front end of DE queue. : F = F+1

(3) Insertion at Front end of DE queue. F = F-1 {F=0 overflow}

(4) Deletion at Rear end of DE queue. R = R-1 . {F=R underflow}

(1) Insertion at Rear end of DE queue. :-

```
void insert()
{
    int element;
    if(R==MAX-1)
     printf("queue is overflow");
   else
   {
      R=R+1;
    if(Front==-1).
                Front=0;
```

```
        printf ("Enter a value");
        &Canf ("%d", &element);
      DE_queue [R] = element;
    }
  }
```

② Deletion at Front End of DE queue.

```
Void delete()
{   int element;
    if (F == -1 || F>R)
      printf ("Underflow");
    else
    {   element = DE_queue [F];
        printf ("%d is deleted", element);
        F = F+1;
    }
}
```

③ Insertion at Front End of DE queue.

```
Void Insert()
{ int element;
  if (Front == 0)
     printf ("overflow");
  else
  { printf ("Enter a element");
```

```
scanf("%d", &element);
Front = Front - 1;
DE_queue [Front] = element;
```

④ Deletion at Rear End of DE queue.

```
Void delete()
{
  int element;
  if (Front == Rear)        // (Rear == -1 or Front > Rear)
    printf("Underflow");
  else
  {
    element = DE_queue [Rear];
    Rear = Rear - 1;
    printf("%d is deleted", element);
  }
}
```

Application of Queue:-

(i) When a resource is shared among multiple Consumers. Example include cpu scheduling.

(ii) When data is transferred asynchronously (data not necessarily received at same rate as sent) b/w two processes.

(iii) All type of customer service (like railway ticket reservation) Center softwares are designed using queues to store customers information.

Priority Queue :- Priority queue is a data structure in which element can be stored as per their priorities. and we can remove the elements according to their priority

* **Process :-** Process is defined as whose execution is started but not yet to be completed.

* **Process Control block (PCB) :-** PCB are represented in operating system. the main function of PCB to maintain the information of each process.

Program status	Program pointer
Program Counter	Program No.
Program ID	
PCB – P_0 PCB – P_1 ! ! PCB – P_4